

A Minor Project

On

TOPIC

File Management System

**Submitted in Partial fulfillment for the award of
Bachelor of Engineering**

Submitted to

**RAJIV GANDHI PROUDYOGIKI VISHWAVIDHYALAYA
BHOPAL (M.P.)**



Submitted by

Shivam kumar-0133CI231262

Satyam Yadav-0133CI231251

Sonu kumar-0133CI231276

Sohil Akhtar-0133CI231273

**Under the supervision of
(Prof. Pawan Tiwari)**



DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY
SAGAR INSTITUTE OF RESEARCH & TECHNOLOGY, BHOPAL
SESSION -June-July



DECLARATION

I hereby declare that the Project synopsis entitled “**File management System**” is My own work conducted under the supervision of **Pawan Tiwari, Bhopal, Computer Science & Information Technology at Sagar Institute of Research & Technology, Bhopal.**

I further declare that to the best of my knowledge this report does not contain any part of work that has been submitted for the award of any degree either in this institute or in other institute without proper citation.

Project Coordinator
Prof Pawan Tiwari

Students Names and Enrollment No
Shivam kumar - 0133CI231262
Satyam Yadav - 0133CI231251
Sonu Kumar - 0133CI231276
Sohil Akhtar - 0133CI231273



SAGAR INSTITUTE OF RESEARCH & TECHNOLOGY, BHOPAL

**DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION
TECHNOLOGY**

LIST OF TABLES

<u>S.No.</u>	<u>HEADING</u>	<u>PAGE No.</u>



SAGAR INSTITUTE OF RESEARCH & TECHNOLOGY, BHOPAL

**DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION
TECHNOLOGY**

TABLE OF CONTENTS

S.No.	HEADING	PAGE No.
1.	Introduction	
1.1	Purpose/objective	
1.2	Scope	
1.3	Software Requirements	
1.4	Hardware Requirements	
2	Data Collected	
2.1	Analysis	
2.2	Literature Survey	
2.3	Software Model	
3.	Designing	
3.1	Data Design(“ER” if applicable)	
3.2	User Interface Architecture if Designed	
3.3	Use Case if Designed	
3.4	DFD/SEQ. Diagram/Activity Diagram if Designed	
4	Cost Estimation	
5	Limitations & Future Enhancements	
6	Conclusion	
7	Bibliography & References	

INTRODUCTION

A File Management System (FMS) or Digital Locker System is a structured software application designed to store, organize, retrieve, secure, and manage user files and documents efficiently. In modern digital environments, the volume of digital data is growing exponentially, and institutions, organizations, and individuals require simple yet reliable platforms to manage files. Traditional file storage methods such as local drives, pen drives, and offline folders often lead to file duplication, misplacement, security risks, and difficulty in retrieval. A cloud-based File Management System solves these challenges by offering centralized storage, quick search functionality, user authentication, secure access, and smooth file sharing.

The Digital Locker Suite developed in this project is a web-based platform built using **React (frontend)**, **Vite (build tool)**, **Tailwind CSS (styling)**, **TypeScript**, and **Supabase (authentication + database + storage)**. It provides an easy-to-use interface where users can upload, download, delete, preview, and share files securely. The system also includes features like folder categorization, metadata storage, access control, and version management depending on the use case. The platform aims to replace traditional offline file storage with a more modern, fast, and secure cloud solution.

This project focuses on providing a real-world system that can be used by students, teachers, organizations, or corporate teams to store documents and manage them digitally. The system can further be extended with mobile support, AI-powered search, OCR scanning, and advanced security mechanisms. The report explains the entire development lifecycle, including requirement analysis, software models, data design, system architecture, DFDs, ER diagrams, cost estimation, implementation, and future enhancements.

1.1 Purpose / Objective

The main purpose of developing the File Management System is to offer a centralized, secure, and easy-to-use digital storage platform. The key objectives are:

1. To Provide Secure Digital Storage

Users can store their files in the cloud securely with role-based authentication powered by Supabase. Each user gets secure access to their documents without risk of unauthorized file access.

2. To Enable Easy File Upload, Download, and Management

The system allows uploading files of various formats (PDF, images, documents, media). Users can preview and download files anytime using an intuitive interface.

3. To Improve File Organization

Folders, tags, and metadata help categorize files, reducing time spent searching. This makes information retrieval quick and efficient.

4. To Facilitate File Sharing

Users can share files with other users or via secure share links. Access can be restricted to view-only or download permission.

5. To Ensure Accessibility from Anywhere

Because it is a web-based system, users can access their files from any device with internet connectivity.

6. To Reduce Physical Storage Dependency

Digital storage eliminates the need for USB drives, system folders, and physical document storage.

7. To Support Institutional File Management

Colleges or offices can use this system to manage documents such as assignments, certificates, training files, and internal documents.

1.2 Scope

The scope of this project includes designing and developing a fully functional file management web application with the following capabilities:

In-Scope Features

- ✓ User authentication (login/signup)
- ✓ Uploading and storing files in cloud storage
- ✓ Organized folder-based structure
- ✓ Preview and download functionalities
- ✓ Responsive UI using React and Tailwind
- ✓ Secure database management using Supabase
- ✓ Role-based permissions (Admin/User)
- ✓ Search functionality for quick retrieval
- ✓ Deleting and restoring files
- ✓ Maintaining metadata for each file

Out-of-Scope Features (For Future Work)

- ✗ AI-based document scanning
- ✗ OCR text extraction
- ✗ Real-time collaborative editing
- ✗ Offline file sync
- ✗ Multi-cloud file replication
- ✗ End-to-end file encryption

The current scope focuses on developing a secure, stable, and user-friendly Digital Locker System suitable for educational institutions, personal use, or small organizations.

1.3 Software Requirements

1. Software Requirements (System Side)

- Operating System: Windows 10/11, Linux, or macOS
- Frontend Framework: React + Vite
- Styling: Tailwind CSS
- Programming Languages: TypeScript, JavaScript
- Database & Authentication: Supabase
- Storage: Supabase Storage Bucket (Cloud Storage)
- Build Tool: Vite
- Package Manager: npm or bun
- IDE: VS Code
- Browser: Chrome, Edge, Firefox

2. Software Requirements (User Side)

- Latest web browser
- Working internet connection
- User account to access the portal

1. Development Environment

- Processor: Intel i5 or higher
- RAM: Minimum 8 GB
- Storage: Minimum 20 GB free
- GPU: Not required, but helpful for faster builds

2. Deployment Server

- Cloud VM (AWS / Azure / Render / Supabase hosting)
- Minimum configuration:
 - 4 GB RAM
 - 2 vCPU
 - 50–100 GB SSD
- Reliable Internet (high uptime)

3. Client-Side Hardware

- Any mobile, laptop, or desktop with internet
- Minimum 2 GB RAM
- Browser support required

2.1 System Overview

The Digital Locker Suite is a cloud-based file storage and management system that allows verified users to upload, store, organize, preview, share, and manage digital files. It integrates with **Supabase** for authentication, database operations, and file storage services. The system is accessible through a web browser and provides a responsive, user-friendly, and secure environment for digital document handling.

This SRS defines the complete set of requirements necessary for the successful design, development, implementation, and maintenance of the Digital Locker Suite. It outlines how the system will manage user data, handle file storage, ensure data privacy, and provide seamless interactions between users and the system interface.

It also specifies the functional workflows such as authentication, file upload, retrieval, deletion, and categorization, ensuring that every operation aligns with user expectations and system constraints.

Additionally, this SRS serves as a communication bridge between developers, stakeholders, and users, ensuring that all parties share a common understanding of the system's goals and capabilities. It provides a clear roadmap for development teams to follow, minimizes ambiguity, and helps maintain consistency throughout the software lifecycle.

The SRS also includes system performance expectations, security protocols, design limitations, and integration requirements, making it a comprehensive foundation for building a secure, scalable, and user-friendly file management system.

2.1 System Overview

The Digital Locker Suite is a cloud-based file storage and management system that allows verified users to upload, store, organize, preview, share, and manage digital files. It integrates with **Supabase** for authentication, database operations, and file storage services. The system is accessible through a web browser and provides a responsive, user-friendly, and secure environment for digital document handling.

The system is designed to replace traditional offline file storage methods with a centralized, secure, and easily accessible digital repository. It ensures that users can manage their documents from any location and device without the risk of data loss. With structured folder organization, metadata management, and role-based access control, the system provides an efficient workflow for handling personal and institutional documents.

The Digital Locker Suite uses a modular architecture built with React and TypeScript, ensuring maintainability and scalability as the project grows. Supabase acts as the backend service, offering real-time data synchronization, user authentication, and cloud storage, making the system capable of supporting multiple concurrent users with high reliability.

The interface is optimized with Tailwind CSS for speed and responsiveness, ensuring smooth interaction across desktops, tablets, and mobile devices. The system also incorporates search, filtering, activity tracking, and secure sharing options to enhance the user experience.

Overall, the Digital Locker Suite serves as a comprehensive and modern solution for document preservation, secure access, and streamlined file management in academic, professional, and personal environments.

2.2 User Characteristics

The Digital Locker Suite is designed for multiple categories of users, each having specific roles, access rights, and responsibilities within the system. Understanding these user types is essential for defining security policies, workflow design, and system permissions. The major user categories include.

The system supports multiple user categories with different roles, privileges and responsibilities. Defining these user types precisely helps set access control, workflows, auditing and security policies. The recommended model is **role-based access control (RBAC)** with optional attribute-based rules (ABAC) for fine-grained decisions (e.g., time-of-day, IP, device posture).

1. Admin

- Manages users
- Manages folder structure
- Monitors storage usage
- Controls file permissions

2. Registered User

- Upload files
- Delete/restore files
- Create and manage folders
- Search and preview documents
- Share files with others

2.3 Functional Requirements

FR-01 User Authentication

- User can **sign up** using email & password.
- User can **log in** using Supabase Auth.
- Password recovery using email OTP.

FR-02 File Upload

- User uploads a file (PDF, image, DOCX, etc.).
- Metadata is stored in Supabase database (file name, size, created_at, uploaded_by).
- File is saved in **Supabase Storage Bucket**.

FR-03 File Preview

- System should preview PDF, images, and text files directly on UI.
- Unsupported files show generic placeholder.

FR-04 File Download

- User can download their uploaded files.
- Log is stored (optional future enhancement).

FR-05 Folder Management

- Users can:
 - Create folders
 - Rename folders
 - Delete folders
 - Move files between folders

FR-06 Search Functionality

- Search by:
 - File name
 - Metadata
 - Upload date
 - File type

FR-07 File Sharing

- User can generate a share link.
- Share link may have:
 - Expiry date
 - View/download permissions
 - Optional password

FR-08 Trash / Restore

- Deleted files move to Trash instead of permanent deletion.
- User can restore or permanently delete files.

FR-09 User Profile Management

- Update name
- Update password
- View storage usage

2.4 Functional Requirements

FR-01 User Authentication

- User can **sign up** using email & password.
- User can **log in** using Supabase Auth.
- Password recovery using email OTP.

FR-02 File Upload

- User uploads a file (PDF, image, DOCX, etc.).
- Metadata is stored in Supabase database (file name, size, created_at, uploaded_by).
- File is saved in **Supabase Storage Bucket**.

FR-03 File Preview

- System should preview PDF, images, and text files directly on UI.
- Unsupported files show generic placeholder.

FR-04 File Download

- User can download their uploaded files.
- Log is stored (optional future enhancement).

FR-05 Folder Management

- Users can:
 - Create folders
 - Rename folders
 - Delete folders
 - Move files between folders

FR-06 Search Functionality

- Search by:
 - File name
 - Metadata
 - Upload date
 - File type

FR-07 File Sharing

- User can generate a share link.
- Share link may have:
 - Expiry date
 - View/download permissions
 - Optional password

FR-08 Trash / Restore

- Deleted files move to Trash instead of permanent deletion.
- User can restore or permanently delete files.

FR-09 User Profile Management

- Update name
- Update password
- View storage usage

2.5 Interface Requirements

User Interface (React + Tailwind)

- Dashboard Page
- File Upload Page
- Folder View Page
- Preview Modal
- Login & Signup Pages
- User Settings Page

Hardware Interface

- Any computer or smartphone with internet
- No special hardware needed

Software Interface

- Browser (Chrome, Firefox, Edge)
- Supabase APIs

2.6 Database Requirements (Based on your project files)

Tables:

1. users (Supabase default)

- id
- email
- created_at

2. files

- id
- file_name
- file_url
- folder_id
- created_at
- uploaded_by

3. folders

- id
- folder_name
- parent_folder_id
- created_at
- created_by

3. shared_files (optional future enhancement)

4. id
5. file_id
6. shared_with
7. expires_at
8. permissions

SECTION 2.1 ANALYSIS

The Analysis phase determines system components, workflows, user interactions, and data flow patterns.

A. Requirement Analysis

Users:

- Admin
- Registered User

System Components:

1. React Frontend
2. Supabase Auth
3. Supabase Database (PostgreSQL)
4. Supabase Storage Bucket
5. REST API / Supabase Client SDK

B. Workflow Analysis

1. File Upload Workflow

1. User selects file
2. File sent to Supabase Storage
3. URL returned
4. Metadata stored in database
5. UI updates file list

2. Login Workflow

- Enter email/password → Supabase verifies → returns session token
→ user navigates to dashboard

3. File Preview Workflow

- System fetches file URL → opens viewer in modal

4. File Sharing Workflow

- Unique share token generated → Saved in DB → User gets link

C. Security Analysis

- Authentication: Supabase Auth
- Database row-level security (RLS) recommended
- File access controlled using signed URLs

SECTION 2.2 LITERATURE SURVEY

A comparison with existing systems:

1. Google Drive

- Highly scalable, real-time collaboration
- Expensive for custom hosting

2. Dropbox

- Fast sync
- Not customizable for institutions

3. SharePoint

- Enterprise level
- Complex setup

4. Traditional File System

- Not cloud accessible
- High data loss risk

5. Proposed Digital Locker System

- Lightweight
- Built for educational/institutional use
- Easy deployment
- Cost-effective

SECTION 2.3 SOFTWARE MODEL

A. Architecture Model (MVC + Cloud Model)

1. View Layer (React + Tailwind)

- UI components
- Modal dialogs
- Navigation

2. Controller Layer

- API calls
- State management
- File validation

3. Model Layer (Supabase Database)

- Users
- Files
- Folders

B. Deployment Model

- Frontend deployed on: Netlify / Vercel / Supabase hosting
- Database: Supabase PostgreSQL
- Storage: Supabase Storage

B. UML Diagram (Text Description)

Use Case Diagram

- Actors: User, Admin
- Actions: Upload, download, create folder, delete, preview, share

Sequence Diagram

- User → React → Supabase Storage → DB → React

Designing (Overall System Design)

The design phase describes how the system is structured internally. It includes the data architecture, user interface layout, workflows, entity relationships, and diagrams representing functional flow. This project uses a **component-based UI (React)** and **cloud-based backend architecture (Supabase)**.

The design focuses on simplicity, modularity, maintainability, and secure file operations. Every feature—from uploading files to creating folders—is implemented as a separate module/component.

This section includes:

1. Data Design (ER Model)
2. User Interface Architecture
3. Use Case Diagrams
4. DFD (Data Flow Diagram)
5. Activity Diagram
6. Sequence Diagram

8.1 Data Design (ER Model)

The project uses Supabase's PostgreSQL database to store metadata and user information. Actual files are stored in Supabase Storage Bucket, while the database stores only URLs, metadata, folder structure, and sharing information.

3.1.1 Entities and Attributes

1. User

<u>Attribute</u>	<u>Description</u>
<u>user_id (PK)</u>	<u>Unique ID of user (UUID)</u>
<u>email</u>	<u>User email (unique)</u>
<u>password_hash</u>	<u>Stored by Supabase Auth</u>
<u>created_at</u>	<u>Account creation timestamp</u>

<u>Attribute</u>	<u>Description</u>
<u>role</u>	<u>admin / regular user</u>

2. File

<u>Attribute</u>	<u>Description</u>
<u>file_id</u> (PK)	<u>Unique file ID</u>
<u>file_name</u>	<u>Original uploaded file name</u>
<u>file_url</u>	<u>Supabase storage URL</u>
<u>size</u>	<u>File size (bytes)</u>
<u>type</u>	<u>File type (PDF, image, doc, etc.)</u>
<u>folder_id</u> (FK)	<u>Folder containing the file</u>
<u>created_at</u>	<u>Upload timestamp</u>
<u>uploaded_by</u> (FK)	<u>User who uploaded the file</u>
<u>status</u>	<u>active / deleted (trash)</u>

3. Folder

<u>Attribute</u>	<u>Description</u>
<u>folder_id</u> (PK)	<u>Unique folder ID</u>
<u>folder_name</u>	<u>Name of folder</u>
<u>parent_folder_id</u>	<u>Hierarchy support</u>
<u>created_by</u>	<u>Owner of the folder</u>
<u>created_at</u>	<u>Creation timestamp</u>

4. Shared Files

<u>Attribute</u>	<u>Description</u>
<u>share_id</u> (PK)	<u>Unique sharing record</u>
<u>file_id</u> (FK)	<u>Shared file</u>
<u>shared_with</u>	<u>Email or user_id</u>
<u>permissions</u>	<u>view / download</u>
<u>expires_at</u>	<u>Expiry date of link</u>

3.2.1 Main UI Components

1. Login Page

- Email & password fields
- Integration with Supabase Auth
- Error handling & validation

2. Dashboard

- Shows recently uploaded files
- Storage usage stats
- Quick upload button

3. Folder View

- Sidebar: Folder tree
- Main panel: Files inside selected folder
- Breadcrumb navigation

4. File Upload Component

- Drag & drop uploader
- Accepts multiple file types
- Shows upload progress

5. File Preview Component

- PDF viewer
- Image preview
- Text preview
- Metadata display

6. Trash Module

- Shows deleted files
- Options: restore or permanently delete

3.2.2 UI Navigation Flow

1) Start / Authentication

- User opens the app
- Login / Signup / Forgot Password
- Successful login → Dashboard

2) Dashboard (Main Navigation Hub)

From Dashboard, user can go to:

- **Files**
- **Upload**
- **Search**
- **Shared with Me**
- **Recent**
- **Trash**
- **Settings**
- **Admin Panel** (Admin only)

3) Files Section

- Shows all files & folders
- User can:
 - Open folder
 - Preview files
 - Download
 - Share
 - Delete → Trash
 - Restore from Trash
 - Create New Folder

4) Upload Flow

- Click **Upload**
- Select files / drag & drop
- Upload progress
- Add metadata (title, tags)
- File appears in folder

5) Sharing Flow

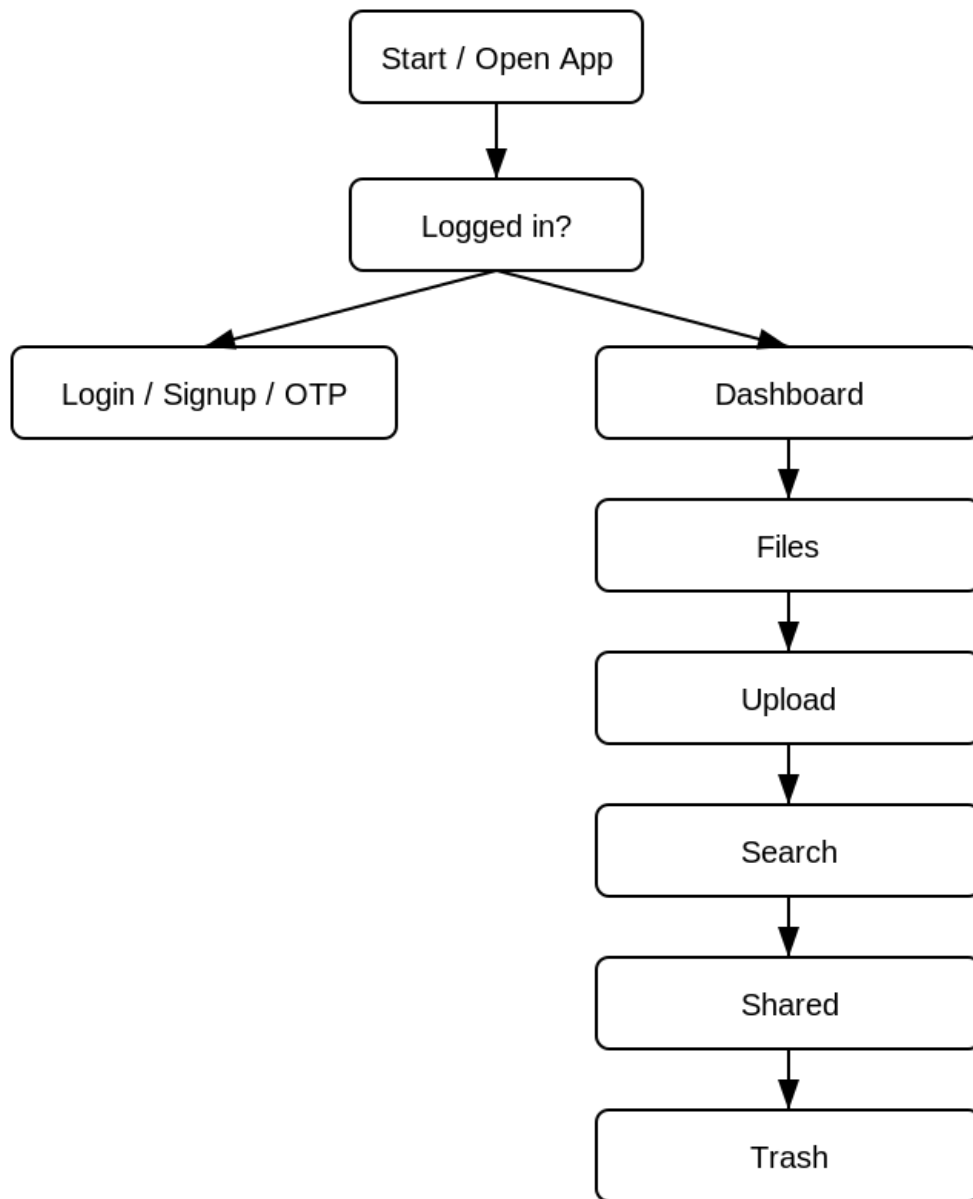
- Select a file → Share
- Add user email / generate link
- Set permission: View / Edit
- Optional: Expiry date
- File moves to **Shared with Me** for receivers

6) Search Flow

- Type keyword in search bar
- Results show:
 - Files
 - Folders
 - Tags
- User can filter by file type, date, owner

7) Trash Flow

- Deleted items stored temporarily
- User can:
 - Restore
 - Permanently delete



3.2.3 Technology Architecture

Frontend:

- React
- Tailwind CSS
- Vite
- TypeScript

Backend:

- Supabase Auth (JWT)
- Supabase Database (PostgreSQL)
- Supabase Storage (file storage)

Communication:

- Supabase Client SDK
- REST + RPC

3.3 Use Case Design — Digital Locker Suite

Actors

- **User** — a registered individual who stores, organizes, views, shares and manages their own documents inside the Digital Locker. May be further subdivided into: Registered User, Guest (limited access), and Power User (advanced privileges such as bulk actions).
- **Admin** — system administrator responsible for managing user accounts, global folder structures, storage quotas, permissions, monitoring usage, and handling escalations.
- **System (Supabase backend)** — the backend platform that provides authentication, database, storage (object/file) services, realtime

events, functions/triggers, and security rules. The System actor performs automated tasks such as verification, indexing, thumbnails generation, audit logging, and notifications.

Use Case Catalogue (high level)

1. User Registration & Authentication (Login/Logout/Forgot Password)
2. Profile Management
3. File Upload / Replace / Versioning
4. Create / Rename / Delete / Move Folders
5. File Preview / Download
6. File Search & Filters
7. Share File / Manage Share Links & Permissions
8. File & Folder Permissions (ACL)
9. Trash / Restore / Permanent Delete
10. Audit Logs / Activity Feed
11. Admin: User Management (create, suspend, delete)
12. Admin: Storage Quota & Reporting
13. Admin: Access Policy & Role Management
14. System: Background Processing (thumbnailing, virus-scan, OCR, indexing)
15. Notifications (email/in-app)
16. Backup & Restore

Detailed Use Cases

Use Case: User Registration & Authentication

- **Primary actor:** User
- **Goal:** Securely register and authenticate to the system.
- **Preconditions:** User has an email address or third-party identity provider account (Google, GitHub, etc.). Supabase auth is available.

- **Main success scenario:**
 1. User opens the registration page.
 2. User provides required details (email, password, name) or uses OAuth provider.
 3. System (Supabase) validates input, creates account, sends verification email.
 4. User verifies email, signs in.
 5. System creates a default user workspace and sets default quotas/permissions.
- **Extensions / alternate flows:**
 - 2a. If email already exists → system returns error and prompts for password reset.
 - 3a. If CAPTCHA fails or email cannot be sent → show friendly error and retry option.
- **Postconditions:** Account created (verified or pending) and workspace provisioned.

Use Case: File Upload (single / bulk)

- **Primary actor:** User
- **Goal:** Upload files to user workspace while validating type/size and creating metadata entries.
- **Preconditions:** User is authenticated; storage available in Supabase storage bucket; file size within allowed limit.
- **Main success scenario:**
 1. User selects files (single or multiple) and target folder.

2. Client calculates metadata (name, MIME type, size, checksum) and optional tags.
 3. Client requests an upload session/token from System (Supabase Storage signed URL or direct upload key).
 4. File is uploaded to Supabase storage; System stores metadata record in database and links to storage path.
 5. System triggers background jobs: thumbnail generation, OCR/indexing, antivirus scan.
 6. System returns success and updates UI; user sees uploaded files with progress and status.
- **Exceptions:**
 - File exceeds quota → show quota exceeded error and suggest upgrade.
 - Network interruption → resume capability or retry.
 - Unsupported MIME → reject with explanation.
 - **Postconditions:** File stored, metadata saved, indexing started.

Use Case: File Preview / Download

- **Primary actor:** User (or Guest with a share link)
- **Goal:** Preview document inline or download a copy with respect to permissions.

Use Case: Share File (Link + Permissions)

- **Primary actor:** User
- **Goal:** Share files with other users or outside via secure links that can be password-protected, expire, or be limited to view/comment/download.

- **Preconditions:** User owns file or has sharing rights.
- **Main flow:**
 1. User selects file(s) → Share.
 2. User picks sharing mode: internal user invite (by email/username) or public link.
 3. User sets permissions: View / Comment / Download / Edit, optional password, expiration date, and max downloads.
 4. System creates share record, generates secure token (signed URL) and stores metadata.
 5. System sends notifications (email/in-app) if internal invite used.

Extensions:

- User revokes a share → link disabled immediately and access events blocked.
- If recipient is outside and password-protected, they must supply password before preview.

Use Case 1: Upload File

Preconditions

- User must be logged in

Main Flow

1. User selects file →
2. React validates file →
3. File uploaded to Supabase Storage →
4. URL returned from storage →

5. Metadata saved in DB →
6. Updated file list shown

Postcondition

- File available for preview/download

Use Case 2: Create Folder

Flow:

1. User clicks “New Folder”
2. Enters folder name
3. Folder inserted into DB
4. UI refreshes folder tree

Use Case 3: Delete File (Move to Trash)

Flow:

1. User selects file → Delete
2. System updates file status = “deleted” in DB
3. File moves to trash (not removed from storage)

Use Case 4: Share File

Flow:

1. User clicks “Share”
2. Share link generated with permissions
3. Saved in shared_files table
4. User sends link to recipient

Use Case 5: Restore File

Flow:

1. User opens Trash
2. Selects file → Restore
3. Status changes to “active”

Use Case Diagram (Text Design)

DIGITAL LOCKER USE CASE DIAGRAM (PLANTUML + TEXT DESIGN)

Actor — • Authenticate
• Upload Document
• Download Document
• Share Document
• Manage Account

Level 1 DFD (File Management)

External Entities

- **User – Uploads, downloads, edits, deletes files.**
- **Admin – Monitors storage, manages permissions.**
- **Auth Service – Verifies login/identity.**

Processes (1.x)

1.0 User Authentication

- **Input: Login details**
- **Output: Session Token**
- **Data Stores: User DB (verify credentials)**

2.0 File Upload & Storage

- **Input: File + Metadata**
- **Output: Upload Confirmation**
- **Data Stores: File Storage, Metadata DB**
- **Notes: Checks quota, stores file, updates metadata.**

3.0 Search / Preview / Download

- **Input: Search Query / File Request**
- **Output: File Stream / Result List**
- **Data Stores: Metadata DB, File Storage**

4.0 Sharing & Permissions

- **Input: Share Request**
- **Output: Share Link / Access Permission**
- **Data Stores: Share/ACL DB**

5.0 Delete / Restore

- **Input: Delete / Restore Action**
- **Output: Updated Status**
- **Data Stores: Trash Store, Metadata DB**

- Notes: Soft delete → moves file to Trash; restore/ permanently delete.

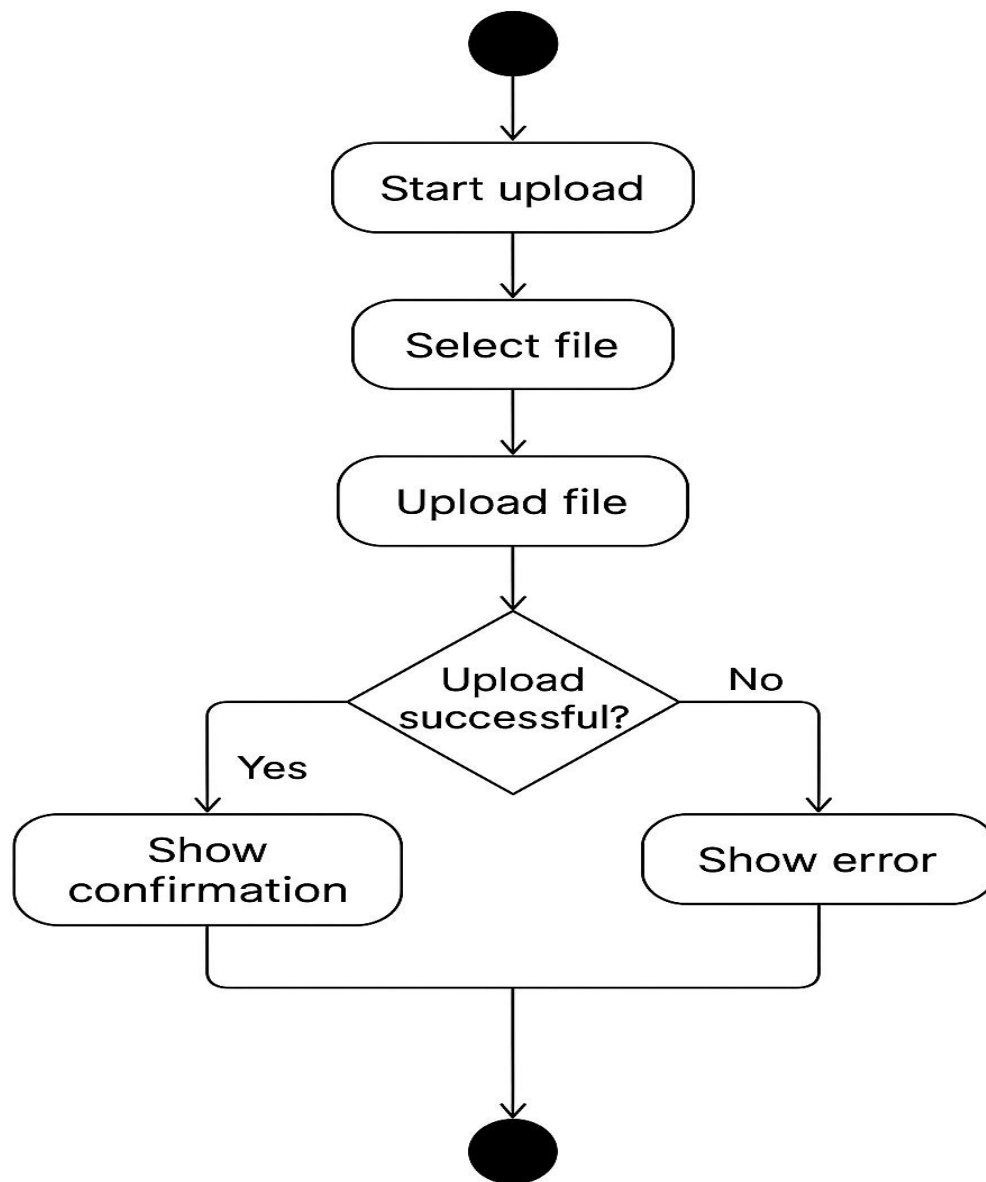
Data Stores (Important Only)

- User DB – login info, roles
- File Storage – actual files
- Metadata DB – names, size, path, timestamps
- Share/ACL DB – permissions, share links
- Trash Store – deleted items until final removal

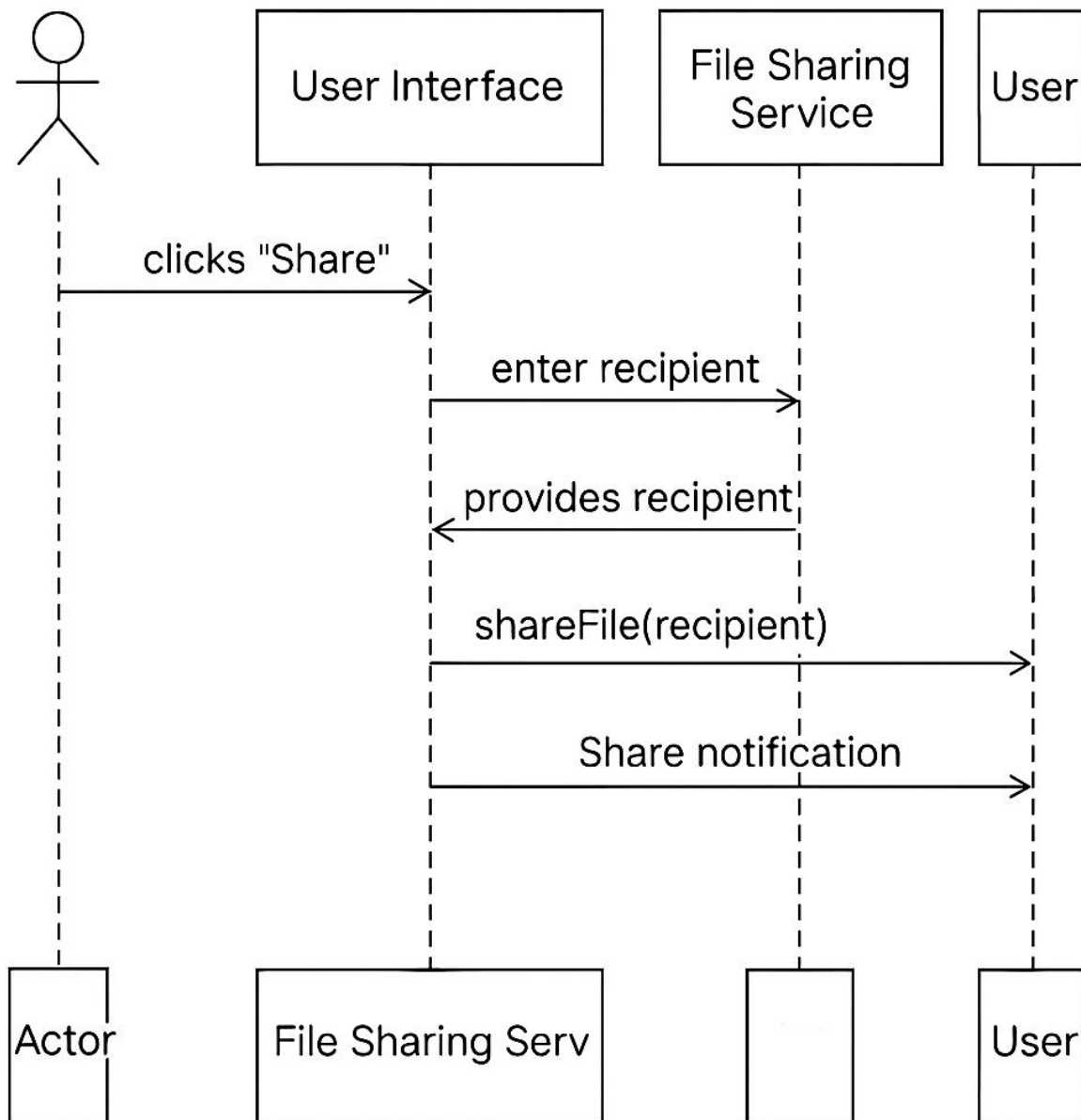
Important Data Flows

- User → 1.0: Login Request
- 1.0 → User: Auth Response
- User → 2.0: File Upload
- 2.0 → File Storage: Store File
- User → 3.0: Download / Preview
- 3.0 → File Storage: Retrieve File
- User → 4.0: Share Request
- User → 5.0: Delete/Restore

Activity Diagram (File Upload)



Sequence Diagram (Share File)



4. Cost Estimation

Cost estimation determines the approximate financial requirements for developing, deploying, and maintaining the Digital Locker Suite (File Management System). Since the system uses modern tools such as React, Vite, Tailwind, TypeScript, and Supabase, many resources are free or open-source, which significantly reduces the development and operational costs.

This section includes:

1. Development Cost
2. Software & Tool Cost
3. Hardware Cost
4. Hosting & Deployment Cost
5. Maintenance Cost
6. Total Estimated Cost Summary

Software & Tools Cost

<u>Software / Service</u>	<u>Type</u>	<u>Cost</u>
<u>React, Vite, Tailwind CSS</u>	<u>Open-source</u>	<u>Free</u>
<u>Visual Studio Code</u>	<u>IDE</u>	<u>Free</u>
<u>Node.js</u>	<u>Runtime</u>	<u>Free</u>
<u>Supabase Free Tier</u>	<u>Database + Storage + Auth</u>	<u>Free (limits apply)</u>
<u>GitHub</u>	<u>Version Control</u>	<u>Free</u>
<u>NPM Packages</u>	<u>Most are free</u>	<u>Free</u>

SECTION 5 — LIMITATIONS & FUTURE ENHANCEMENTS

5. Limitations & Future Enhancements

The Digital Locker Suite (File Management System) is an efficient, cloud-based platform for uploading, storing, organizing, and sharing digital files. While the system successfully covers all essential features, some limitations exist due to technology constraints, free-tier service limits, and the initial project scope. These limitations provide clear opportunities for future improvements and upgrades.

5.1 Limitations

1. Limited Storage Capacity (Free Tier Supabase)

Supabase's free-tier storage offers only 1 GB, which may become insufficient for users uploading large documents, PDFs, and media files. For large-scale usage, upgrading to a paid plan is necessary.

2. No Real-Time Collaboration

Unlike Google Drive, the system does not support real-time editing or simultaneous collaboration. Files must be downloaded, edited, and re-uploaded manually.

3. No File Versioning System

The current version only stores the latest uploaded file. If a user reuploads a file with the same name, older versions are not saved. Version control can be added later.

4. Basic Search Functionality

Search works only on:

- **File name**
- **Basic metadata**

The system does not support full-text search, OCR-based search, or advanced filtering.

5. Limited Sharing Controls

File sharing is functional but lacks features such as:

- **Role-based access**
- **Watermarked preview**
- **Expiry reminders**
- **View-only mode with disabled download**

6. No End-to-End Encryption

Supabase provides secure storage, but the system does not implement custom encryption for extremely sensitive documents. Files are not encrypted with user-specific private keys.

7. Offline Support Not Available

The application is entirely cloud-based. It cannot be used without an active internet connection.

SECTION 6 — CONCLUSION

The **Digital Locker Suite (File Management System)** developed in this project demonstrates how modern cloud-based technologies can be used to build an efficient, secure, and user-friendly platform for managing digital files. With the growing dependence on digital documentation in educational institutions, offices, and personal environments, the need for a centralized, reliable, and easily accessible storage system has become essential.

This project successfully integrates **React** for building a fast and interactive user interface, **Tailwind CSS** for responsive design, **Supabase Authentication** for secure user login and identity management, **Supabase Database** for storing metadata, and **Supabase Storage** for file handling. Together, these technologies provide a lightweight yet powerful solution that simplifies tasks such as uploading, organizing, previewing, sharing, and managing files.

Through the development of this system, we explored concepts such as cloud computing, API-based communication, secure file storage, database design, user interface architecture, and full-stack application development. The project also allowed us to understand real-time challenges such as handling large files, ensuring security, enabling search capabilities, and maintaining an intuitive user experience.

While the current version of the system includes essential features such as file upload, download, folder management, and sharing, there remains plenty of scope for future improvements including advanced role-based permissions, end-to-end encryption, AI-powered search, mobile applications, and sophisticated analytics.

In conclusion, the Digital Locker Suite fulfills the core objectives of providing a secure, accessible, and organized platform for digital file management. It serves as a practical demonstration of combining modern web development tools with cloud backend services, offering a scalable foundation for future enhancements and real-world deployment.

SECTION 7 — BIBLIOGRAPHY & REFERENCES

The following books, websites, documentation pages, and online learning resources were consulted during the development and research phases of the **Digital Locker Suite (File Management System)**. These references helped in understanding technology concepts, implementation techniques, cloud storage methods, and best practices in full-stack development.

7.1 Books & Academic References

1. **Stallings, William.** *Operating Systems: Internals and Design Principles*. Pearson Education.
2. **Pressman, Roger S.** *Software Engineering: A Practitioner's Approach*. McGraw-Hill.
3. **Sommerville, Ian.** *Software Engineering*. Pearson.
4. **Silberschatz, Abraham, Galvin, Peter & Gagne, Greg.** *Operating System Concepts*. Wiley.
5. **Martin Fowler.** *Patterns of Enterprise Application Architecture*. Addison-Wesley.

7.2 Technology & Framework Documentation

- | | | |
|---|----------------------|----------------------------------|
| 1. React | Official | Documentation |
| https://react.dev/ | | |
| 2. Vite | Official | Documentation |
| https://vitejs.dev/ | | |
| 3. Tailwind | CSS | Documentation |
| https://tailwindcss.com/docs | | |
| 4. Supabase | Documentation | (Auth, Database, Storage) |
| https://supabase.com/docs | | |
| 5. Node.js | Official | Documentation |
| https://nodejs.org/en/docs | | |
| 6. TypeScript | | Handbook |
| https://www.typescriptlang.org/docs/ | | |

7. PostgreSQL Official Manual
<https://www.postgresql.org/docs/>

7.3 Research Articles & Online Resources

1. GeeksforGeeks – Articles on File Management Systems
<https://www.geeksforgeeks.org>
2. MDN Web Docs – JavaScript, Web APIs, Authentication
<https://developer.mozilla.org/>
3. FreeCodeCamp – Full-Stack Development Tutorials
<https://www.freecodecamp.org/>
4. W3Schools – Web development references
<https://www.w3schools.com/>
5. DigitalOcean Community – Cloud deployment tutorials
<https://www.digitalocean.com/community>

7.4 Diagrams & Design Tools

1. Draw.io (diagrams.net) – ER Diagrams, DFDs, UML
<https://www.diagrams.net/>
2. Lucidchart UML Tool
<https://www.lucidchart.com/>

7.5 Additional References (Project-Specific)

1. Supabase GitHub Repository
<https://github.com/supabase>
2. NPM Package Documentation (React Hooks, Icons, Utilities)
<https://www.npmjs.com/>
3. GitHub Issues/Discussions – Troubleshooting project setup
<https://github.com/>

THANK YOU